

COMP.SE.140-2025-2026-1 CONTINUOUS DEVELOPMENT AND DEPLOYMENT – DEVOPS

Report on Exercise1

MD SAKIB HASAN

Email: sakib.hasan@tuni.fi

Student ID: gbg859

Repo URL

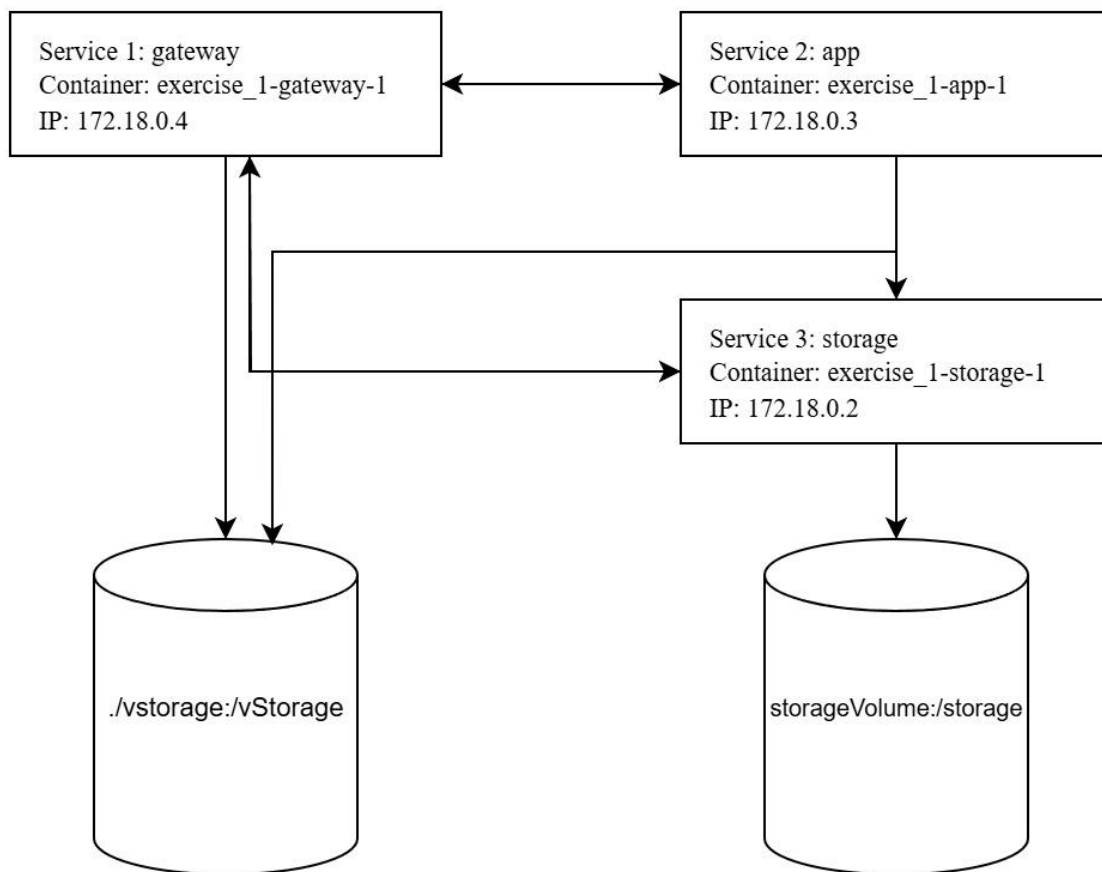
[git@github.com:sakibh20/DevOps-COMP.SE.140-Exercise-1.git](https://github.com/sakibh20/DevOps-COMP.SE.140-Exercise-1.git)

Platform Information

- **Hardware:** Desktop PC
- **OS:** Windows 10, 64-bit
- **Docker Version:** 28.4.0, build d8eb465
- **Docker-Compose Version:** v2.39.2-desktop.1

Diagram

The IPs and Container name may not be same every time we run.



Volumes:

- `./vstorage` → mounted by Gateway and App
- `storageVolume` → mounted by Storage container

Analysis of Status Records

Timestamp1: 2025-09-20T11:56:44Z: uptime 0.00 hours, free disk in root: 966754 MBytes

Timestamp2: 2025-09-20T11:56:44Z: uptime 0.00 hours, free disk in root: 966754 MBytes

What was measured:

Disk space: In the solution I used **df -k** / command inside the container. That command checks free disk space in the container's root filesystem. This shows how much storage is available to that specific container.

Uptime: The Node.js service (**process.uptime()**) and the Python service (**time.time() - start_time**) report the number of hours the process has been running since the container started.

Relevance:

These measurements are real but it is very limited. Free disk in the root of a container is not usually meaningful in production, since containers share storage from the host or volumes.

Uptime only shows how long the container has been running; it does not tell whether the application is healthy or performing well.

What should be done better:

Instead of container root disk space, measure application-specific volumes or host-level disk usage.

For uptime, include service health checks.

Analysis of Storage Solution

There are two implementations of volume in this exercise. vStorage is mounted by host gateway and app services in with local file system (./vstorage) and the data is persistent. But it does not go well with cloud-native architecture. This makes it dependent to host environment. Not good in terms of scalability and portability.

The other approach is using a separate service named storage which runs on its own container. This performs logging with simple HTTP GET and POST protocol. This makes it modular. This adds a bit of complexity in terms of implementation.

Cleaning Up Instructions

- Delete local folder
rm -rf ./vstorage && touch ./vstorage && chmod 666 ./vstorage
- Remove docker volume
docker-compose down -v

Difficulties Faced

- coordinate multiple services writing to the same volume
- keeping logs consistent across vStorage and Storage container.

Main Problems

- Service2 was not writing properly in vStorage. Then found it was because I understood the requests wrong.
- Some error handling was also a bit problematic.
- Calculating free space.